

Function declaration

Functions are declared in your source files or made available by linking precompiled libraries. Declaration syntax of a function is:

```
typedef function_name (parameter-declarator-list);
```

The `function_name` must be a valid identifier. This name is used to call the function.

The type represents the type of function result, and can be any standard or user-defined type. For functions that do not return value, you should use void type. The type can be omitted in global function declarations, and function will assume int type by default.

Function type can also be a pointer. For example, `float*` means that the function result is a pointer to float. Generic pointer, `void*` is also allowed.

Function cannot return an array or another function.

Within parentheses, `parameter-declarator-list` is a list of formal arguments that function takes. These declarators specify the type of each function parameter. The compiler uses this information to check function calls for validity. If the list is empty, function does not take any arguments. Also, if the list is void, function also does not take any arguments; note that this is the only case when void can be used as an argument's type. Unlike with variable declaration, each argument in the list needs its own type specifier and a possible qualifier constant or volatile.

Function prototypes

A given function can be defined only once in a program, but can be declared several times, provided the declarations are compatible. If you write a nondefining declaration of a function, i.e. without the function body, you do not have to specify the formal arguments.

This kind of declaration, commonly known as the function prototype, allows better control over argument number and type checking, and type conversions. Name of the parameter in function prototype has its scope limited to the prototype. This allows different parameter names in different declarations of the same function:

```
/* Here are two prototypes of the same function: */  
int test(const char*)    /* declares function test */  
int test(const char*p)   /* declares the same function  
test */
```

Function prototypes greatly aid in documenting code. For example, the